SOFTWARE ARCHITECTURE DOCUMENT

# Documento de Arquitetura de Software

Plataforma de Controle de Caixa Diário com separação explícita entre os domínios **Ledger** e **Daily Balance**

## METADADOS

| | |
|---|---|
| Documento | Architecture Definition |
| Versão | 1.0 |
| Status | Final |
| Data | 01/04/2026 |

## RESUMO EXECUTIVO

- Dois serviços independentes com integração assíncrona por eventos
- Implementação em Node.js, TypeScript e NestJS
- Clean Architecture, Outbox Pattern e consumidor idempotente
- Foco em custo-benefício, resiliência e clareza de domínio

## STACK

- NestJS
- PostgreSQL
- SQS / ElasticMQ
- ECS Fargate

# Histórico de Revisão

| Data | Versão | Descrição | Autor |
|---|---|---|---|
| 01/04/2026 | 1.0 | Consolidação do documento arquitetural com visão de negócio, diagramas e decisões técnicas | Thiago Ferraz |

# Sumário

**1. Introdução**

1. Finalidade
2. Escopo
3. Visão geral do documento

**2. Drivers de negócio e requisitos**

1. Problema de negócio
2. Capacidades de negócio
3. Requisitos funcionais
4. Requisitos não funcionais

**3. Representação da arquitetura**

1. Visão executiva
2. C4 Model - Nível 1
3. C4 Model - Nível 2

**4. Visão de domínio e visão lógica**

1. Bounded contexts
2. Responsabilidades por domínio

3. Containers e integrações

## 5. Fluxos e decisões

1. Fluxo de lançamento
2. Fluxo de consolidado
3. Trade-offs e tecnologia

## 6. Operação e evolução

1. Observabilidade e segurança
2. Arquitetura de transição
3. Estimativa de custos

# 1. Introdução

## 1.1 Finalidade

Este documento apresenta a arquitetura de software da solução proposta para o desafio de controle de caixa diário. O objetivo é demonstrar, de forma clara e auditável, como a arquitetura atende aos requisitos funcionais e não funcionais com foco em custo-benefício, resiliência e capacidade de evolução.

## 1.2 Escopo

O escopo contempla dois contextos principais de negócio: **Ledger**, responsável pelo registro transacional de débitos e créditos, e **Daily Balance**, responsável por consolidar o saldo diário a partir dos eventos gerados pelo domínio transacional.

O documento cobre visão de negócio, representação arquitetural, separação de domínios, fluxos principais, decisões técnicas, observabilidade, segurança, custos e estratégia de evolução.

## 1.3 Visão geral do documento

> A estrutura segue o padrão de um documento arquitetural formal: parte das necessidades de negócio, apresenta a solução em camadas de abstração e fecha com decisões, atributos de qualidade e roadmap de evolução.

# 2. Drivers de Negócio e Requisitos

## 2.1 Problema de negócio

O comerciante precisa registrar movimentações financeiras ao longo do dia e consultar rapidamente o saldo diário consolidado. Ao mesmo tempo, o sistema deve continuar operando mesmo que o consolidado esteja temporariamente indisponível.

## 2.2 Capacidades de negócio

| Capacidade | Objetivo de negócio | Resultado esperado |
|---|---|---|
| Registrar lançamentos | Garantir controle transacional do caixa | Débitos e créditos armazenados com rastreabilidade |

| Capacidade | Objetivo de negócio | Resultado esperado |
|---|---|---|
| Consultar histórico | Permitir auditoria operacional | Lista de lançamentos por comerciante |
| Consolidar saldo diário | Apoiar tomada de decisão | Saldo diário disponível para consulta |
| Reprocessar eventos | Garantir confiabilidade | Menor risco de perda ou inconsistência |

## 2.3 Requisitos funcionais

**RF-01**
Registrar lançamentos com merchantId, tipo, valor, occurredAt e requestId.

**RF-02**
Permitir consulta de lançamentos por comerciante.

**RF-03**
Consolidar saldo diário com totais de crédito, débito e saldo final.

**RF-04**
Garantir idempotência na entrada para não duplicar lançamentos.

**RF-05**
Garantir idempotência no consumo para não duplicar saldo.

**RF-06**
Atualizar o consolidado a partir de processamento assíncrono por eventos.

## 2.4 Requisitos não funcionais

| Requisito | Diretriz arquitetural |
|---|---|
| Independência entre serviços | Outbox Pattern e fila assíncrona para evitar dependência síncrona entre Ledger e Daily Balance |
| Escalabilidade do consolidado | Consumidor horizontalizável e read model otimizado para consulta |
| Perda máxima tolerada de 5% | Fila durável, retry, DLQ e consumo idempotente visando perda prática próxima de zero |
| Observabilidade | Logs estruturados, métricas de backlog, latência e correlação ponta a ponta |
| Manutenibilidade | Clean Architecture, separação explícita de domínios e testes automatizados |

# 3. Representação da Arquitetura

## 3.1 Visão executiva

A solução foi modelada a partir de capacidades de negócio e não apenas da escolha de tecnologias. O objetivo principal é proteger a operação do comerciante e, ao mesmo tempo, disponibilizar uma visão consolidada do caixa diário.

Figura 1. Visão executiva da solução, destacando ator principal, sistema de controle de caixa e integrações relevantes.

## 3.2 C4 Model - Nível 1

O C4 de contexto posiciona a solução dentro do ecossistema de negócio e mostra como o comerciante interage com a plataforma sem expor detalhes técnicos desnecessários nesse nível.

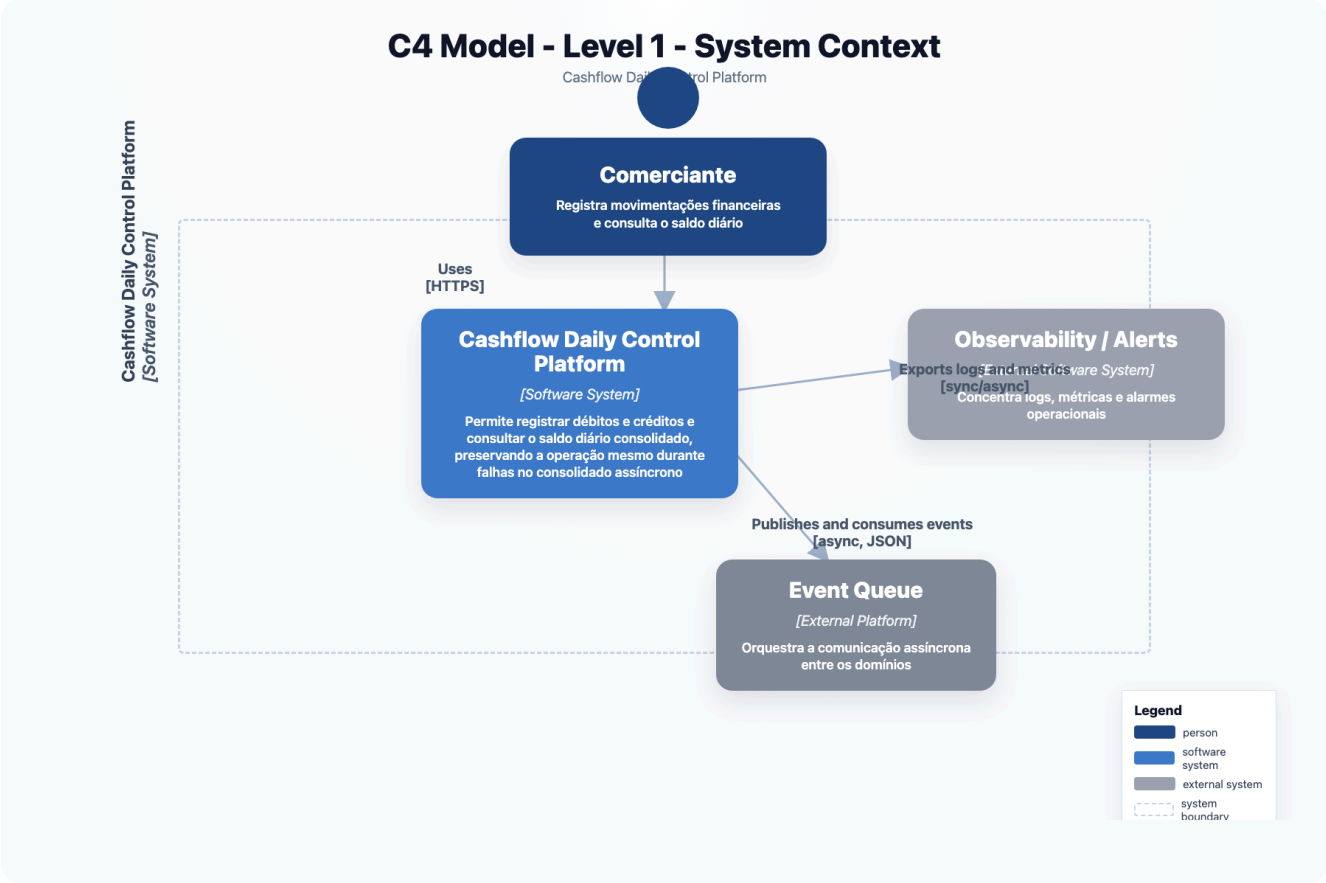


Figura 2. System Context Diagram mostrando o ator principal, o sistema alvo e integrações externas relevantes.

## 3.3 C4 Model - Nível 2

No nível de containers, a arquitetura evidencia a separação entre o contexto transacional e o contexto de consolidação, além dos mecanismos de persistência e mensageria que viabilizam o desacoplamento.

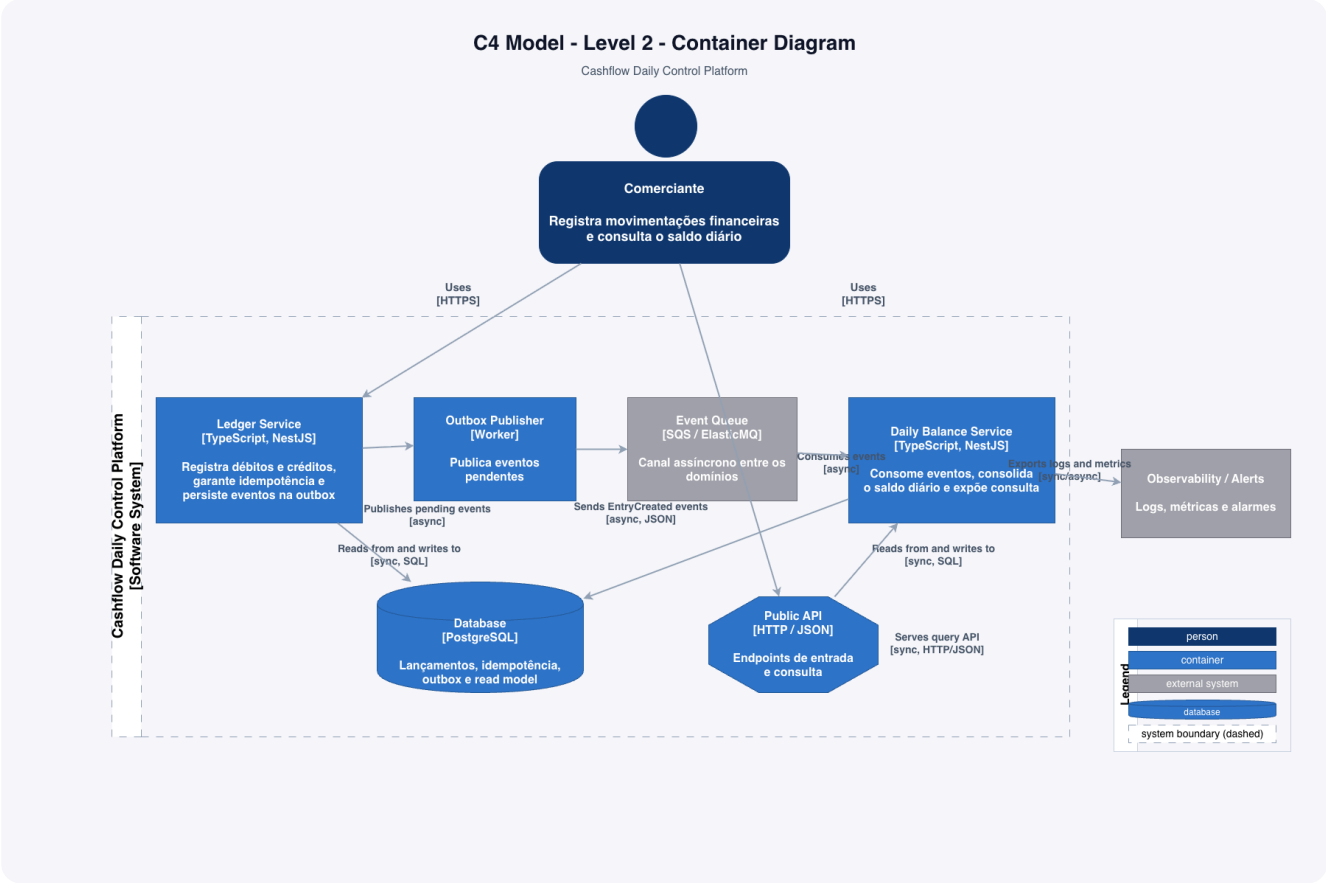


Figura 3. Container Diagram da plataforma, com foco em serviços, mensageria, persistência e observabilidade.

# 4. Visão de Domínio e Visão Lógica

## 4.1 Bounded contexts

O desenho foi estruturado em dois bounded contexts independentes, cada um com responsabilidades e modelos próprios. Essa decomposição é o principal mecanismo para cumprir o requisito de independência entre o serviço transacional e o serviço de consolidado.

Figura 4. Separação entre os contextos **Ledger** e **Daily Balance**, responsabilidades e regras de autonomia.

## 4.2 Responsabilidades por domínio

| Contexto | Responsabilidades | Não faz |
|---|---|---|
| Ledger | Recebe lançamentos, valida regras básicas, persiste histórico imutável, controla idempotência e publica eventos | Não calcula saldo consolidado |
| Daily Balance | Consome eventos, agrega saldo diário, mantém read model e controla idempotência de consumo | Não grava o lançamento original |

## 4.3 Estrutura lógica por contexto

LEDGER

**Domínio transacional**

Organizado em `domain`, `application`, `infrastructure` e `presentation`, com foco em integridade, rastreabilidade e disponibilidade.

DAILY BALANCE

**Domínio derivado**

Estruturado nas mesmas camadas, com foco em agregação, reprocessamento seguro e consulta eficiente do saldo diário.

## 4.4 Diagrama técnico dos containers

A visão técnica complementar explicita como os serviços, a fila e o banco se relacionam em runtime, reforçando que a comunicação entre os domínios ocorre por integração assíncrona.

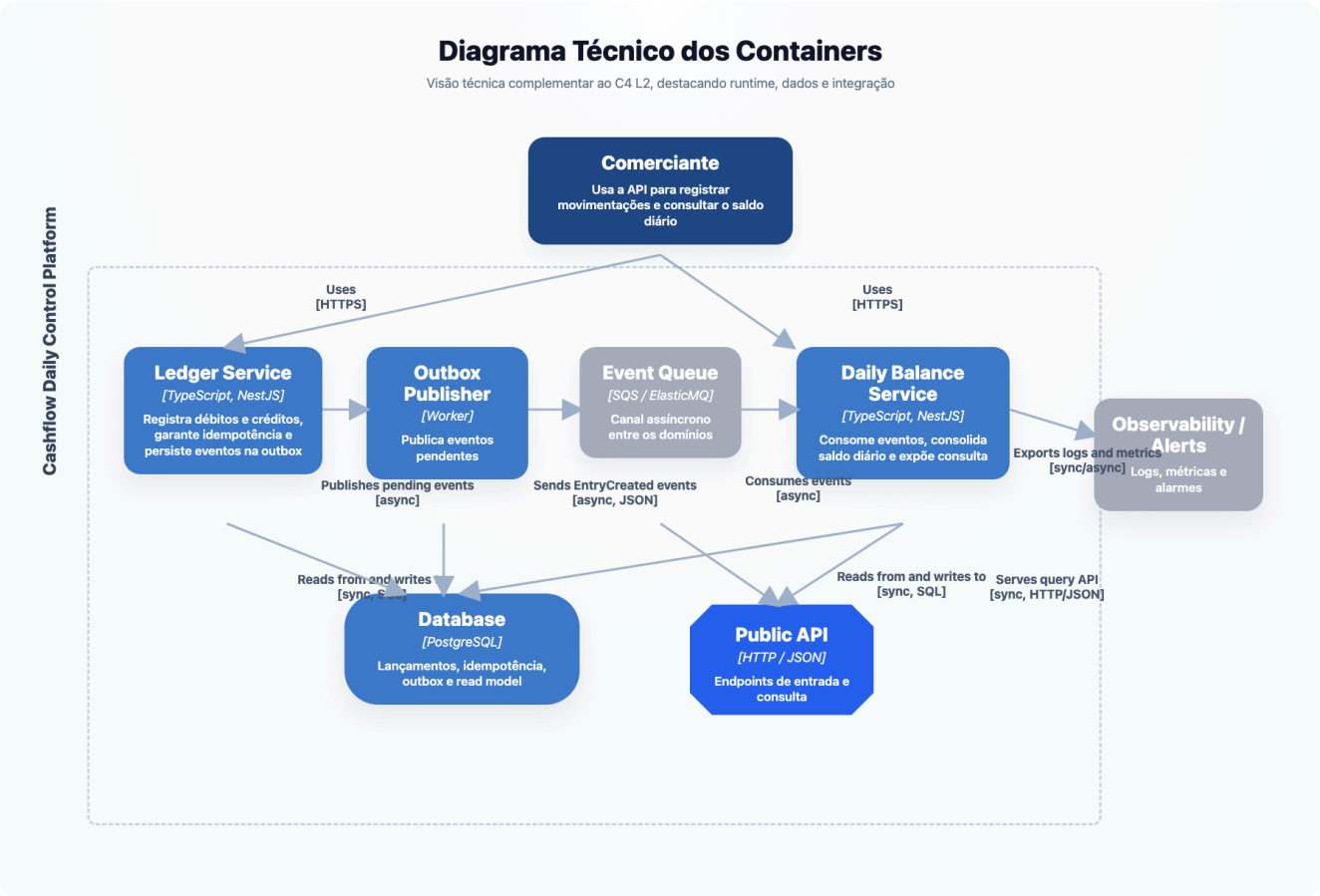


Figura 5. Visão técnica complementar dos containers com runtime, persistência, fila, API e observabilidade.

## 4.5 Responsabilidade dos containers

| Container | Papel | Perspectiva de negócio |
|---|---|---|
| Ledger Service | Registra lançamentos e publica eventos | Garante continuidade da operação transacional |
| Daily Balance Service | Consolida saldo diário | Disponibiliza visão resumida para gestão do caixa |
| PostgreSQL | Persiste fatos e read model | Mantém integridade e rastreabilidade |
| SQS / ElasticMQ | Desacopla os contextos | Evita dependência síncrona entre operação e consolidação |

# 5. Fluxos Principais e Decisões Arquiteturais

## 5.1 Fluxo de lançamento

O fluxo de lançamento garante persistência transacional do fato financeiro e publicação confiável do evento de integração por meio do padrão de outbox.

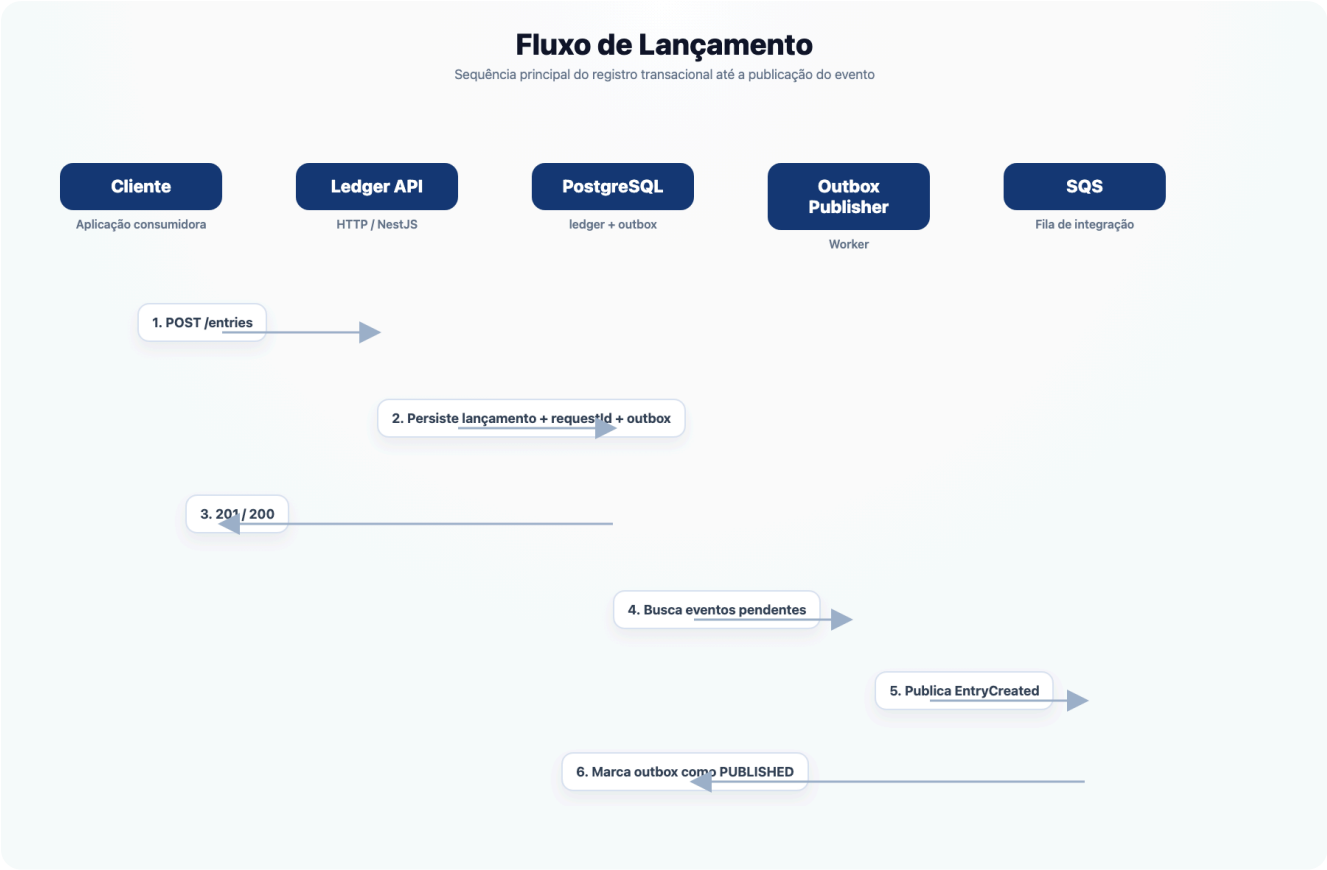


Figura 6. Sequência do registro de lançamento, escrita na outbox e publicação assíncrona do evento.

## 5.2 Fluxo de consolidado

O consolidado consome o evento, valida duplicidade, atualiza o read model e mantém consistência eventual sem impactar o serviço transacional.

Figura 7. Sequência de consumo idempotente e atualização do saldo diário.

## 5.3 Justificativa de tecnologia e trade-offs

| Decisão | Motivação | Trade-off |
|---|---|---|
| Node.js + TypeScript + NestJS | Produtividade, estrutura modular e boa aderência à Clean Architecture | Maior boilerplate em troca de organização e testabilidade |
| PostgreSQL | Consistência transacional e custo-benefício para domínio financeiro | Menor flexibilidade de schema que algumas abordagens NoSQL |
| SQS | Baixo custo, simplicidade operacional e desacoplamento assíncrono | Menos recursos de streaming que Kafka |
| Outbox Pattern | Confiabilidade entre persistência e publicação de eventos | Exige tabela e rotina adicional de publicação |
| Não adoção de RDS Proxy no MVP | Volume do desafio não justifica custo e complexidade extra | Proxy pode ser reavaliado se houver explosão de conexões no futuro |

## 5.4 Decisões técnicas e valor de negócio

| Decisão técnica | Valor gerado |
|---|---|
| Separar Ledger e Daily Balance | Queda do consolidado não paralisa a operação principal do comerciante |
| Read model próprio para saldo diário | Consulta rápida para apoiar a operação do caixa |
| Idempotência na entrada e no consumo | Evita inconsistências financeiras por repetição de mensagens |
| Arquitetura em camadas por domínio | Facilita manutenção, testes e evolução futura |

# 6. Operação, Segurança e Evolução

## 6.1 Observabilidade

**Logs e rastreabilidade**

Logs estruturados com `timestamp`, `level`, `context`, `correlationId` e `requestId` quando aplicável.

**Métricas e alarmes**

Backlog da fila, idade da mensagem mais antiga, falhas do publicador, latência p95 e taxa de erro dos serviços.

## 6.2 Segurança

| Objetivo | Medida adotada ou recomendada |
|---|---|
| Autenticidade do consumo | Guard de API key no consumo e recomendação de JWT/OAuth2 na borda |
| Proteção de dados | TLS obrigatório, criptografia em repouso e segredo fora do código |
| Restrição de acesso | Menor privilégio em IAM, escopo por ambiente e rate limiting |
| Integridade operacional | Idempotência, separação de domínios e reprocessamento seguro |

## 6.3 Arquitetura de transição

O MVP já nasce com fundamentos corretos de domínio, integração e confiabilidade, mas a arquitetura foi desenhada para evoluir sem reescrever as regras centrais do negócio.

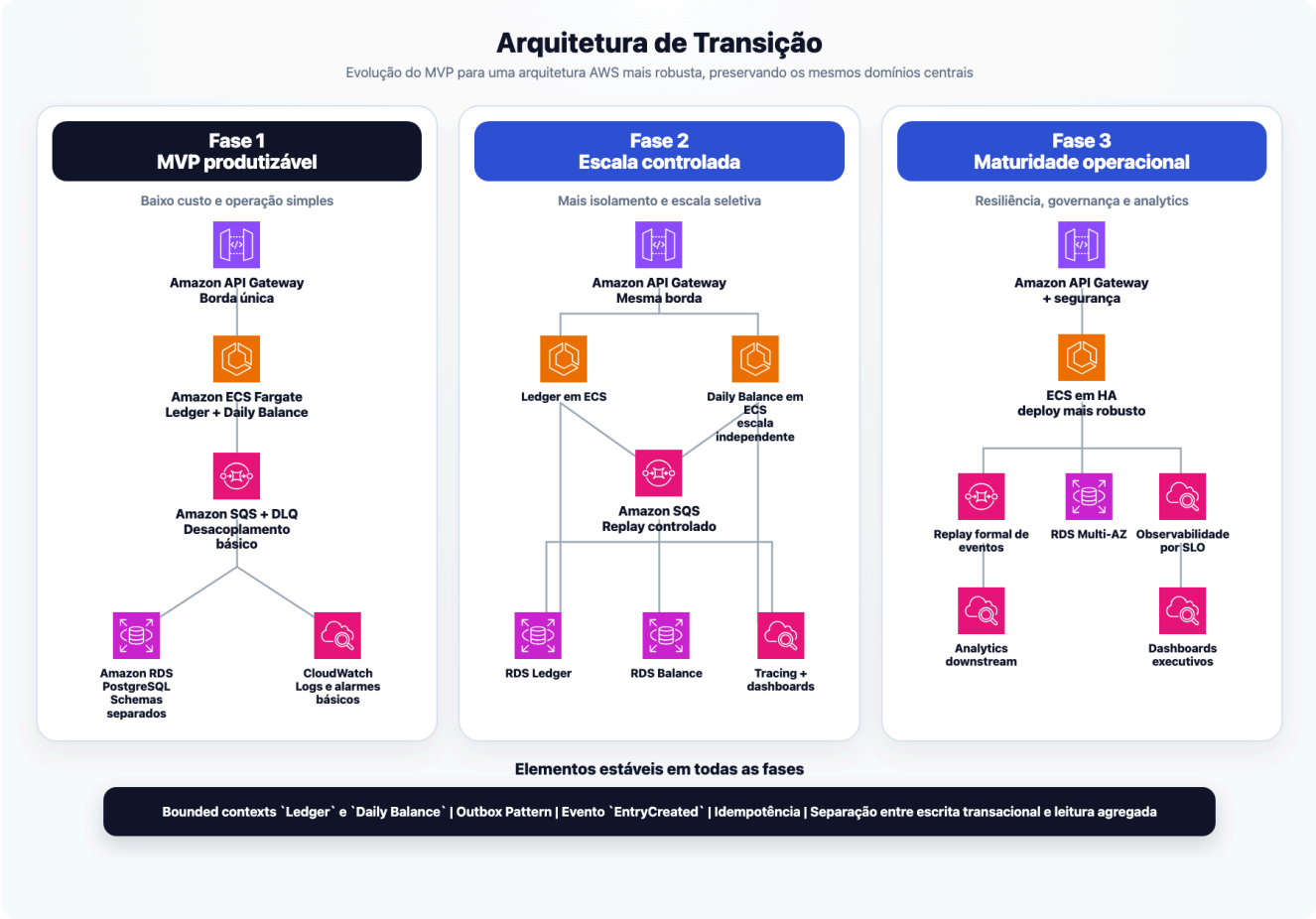


Figura 8. Evolução da solução em AWS, do MVP produtizável até uma operação com maior maturidade e resiliência.

## 6.4 Estimativa de custos

| Componente | Configuração base | Faixa estimada |
|---|---|---|
| ECS Fargate - Ledger | 0.25 vCPU / 0.5 GB | USD 10-18 |
| ECS Fargate - Daily Balance | 0.25 vCPU / 0.5 GB | USD 10-18 |
| RDS PostgreSQL | Instância pequena single-AZ | USD 20-35 |
| SQS | Baixo volume | USD 1-3 |
| CloudWatch | Logs e métricas básicas | USD 5-15 |

> Faixa estimada de MVP: **USD 49-97 / mês**, priorizando serviços gerenciados simples, sem Redis, WAF ou RDS Proxy no escopo inicial.

# 7. Conclusão

A solução proposta equilibra clareza de domínio, confiabilidade operacional e custo-benefício. O desenho evita superengenharia no MVP, mas preserva fundamentos corretos para escalar e amadurecer sem reescrever a lógica principal do negócio.

---

Documento consolidado a partir dos artefatos de requisitos, arquitetura, separação de domínios, trade-offs, observabilidade, segurança, custos e arquitetura de transição existentes no projeto.